

Universidade Federal de Santa Catarina
Centro Tecnológico
Departamento de Informática e Estatística

Searchable Encryption: Busca em Dados Cifrados

Artur Luiz Rizzato Toru Soda, Davi Menegaz Junkes, Matheus Fernandes
Bigolin

Florianópolis
2026

**Artur Luiz Rizzato Toru Soda, Davi Menegaz Junkes, Matheus
Fernandes Bigolin**

Searchable Encryption: Busca em Dados Cifrados

Trabalho apresentado à disciplina INE5429:
Segurança em Computação do Curso de Ciên-
cias da Computação da Universidade Federal
de Santa Catarina, como requisito parcial
para a obtenção de nota na avaliação final.
Professores: Jean Everson Martina e Thaís
Bardini Idalino.

Universidade Federal de Santa Catarina
Centro Tecnológico
Departamento de Informática e Estatística

Florianópolis
2026

Sumário

1	INTRODUÇÃO	3
2	DETALHAMENTO TEÓRICO	4
2.1	Modelo	4
2.2	O Esquema de Song, Wagner e Perrig (2000)	5
2.3	Exemplo Concreto	6
2.4	Propriedades de Segurança	7
2.5	Limitações	7
3	IMPLEMENTAÇÃO	9
3.1	Primitivas e Decisões de Projeto	9
3.2	Pseudocódigo	9
4	RESULTADOS EXPERIMENTAIS	12
4.1	Corretude	12
4.2	Desempenho	12
4.3	Vazamento por Frequência de Acesso	13
5	CONCLUSÃO	15
	REFERÊNCIAS	16

1 Introdução

Repositório do *GitHub*. <<https://github.com/ArturRSoda/searchable-encryption-ine5429>>

A popularização da computação em nuvem transformou como o armazenamento e gerência de dados funcionam, a possibilidade de terceirizar essas funções para servidores remotos reduz custos e esforços de gerenciamento para usuários e empresas, oferecendo uma clara vantagem a soluções internas. Porém, esse modelo introduz um claro problema de segurança: ao delegar a posse dos dados a um provedor sem relação com o usuário, ele perde o controle físico sobre os dados e passa a depender de um servidor que pode não ser plenamente confiável. Nesse sentido, um provedor desonesto, ou um servidor comprometido por um atacante, pode acessar, copiar ou vender essas informações sem que o usuário perceba (WANG et al., 2016).

A defesa mais clara contra esse risco é cifrar os dados antes de enviá-los para o servidor, garantindo confidencialidade. Entretanto, essa solução introduz certas dificuldades na hora de usar e manipular tais dados, quando, uma vez cifrados, perde-se a possibilidade de realizar operações úteis sobre eles, como, por exemplo, a busca por palavras-chaves. A solução trivial para esse problema, em que o usuário baixa todo o conteúdo cifrado, decifra e então realiza a busca é obviamente impraticável, pois anula as vantagens de delegar o armazenamento e apresenta custos altos de processamento. Surge assim, então, a necessidade de uma técnica que concilie a privacidade dos dados e a capacidade de realizar buscas sobre eles (WANG et al., 2016).

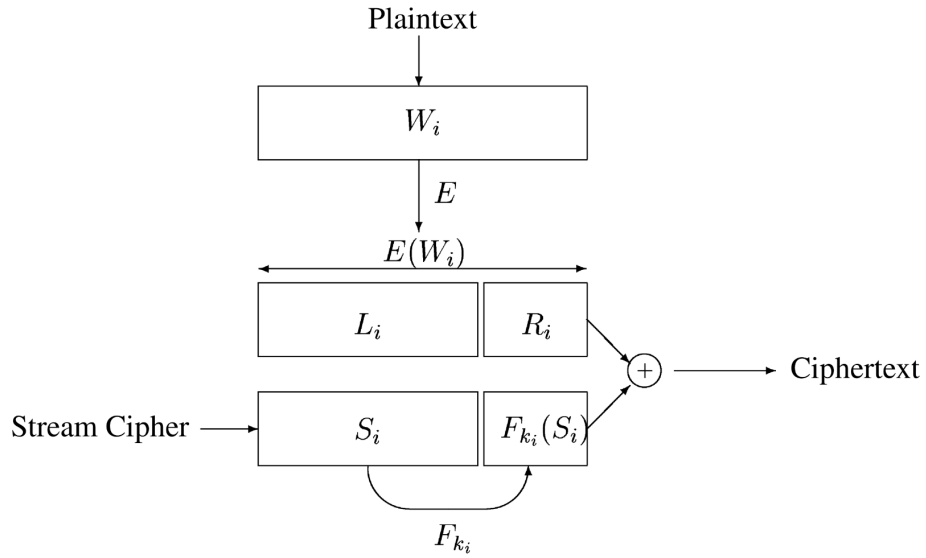
É nesse contexto que surge *Searchable Encryption* (SE), um conjunto de técnicas que permite a busca sobre dados cifrados sem que o conteúdo dos documentos ou a palavra consultada sejam revelados. Dessa forma, o servidor executa a busca e devolve os resultados corretos, mas aprende o mínimo possível no processo. O algoritmo escolhido para este trabalho é o esquema proposto por Song, Wagner e Perrig em *Practical Techniques for Searches on Encrypted Data* (SONG et al., 2000), o primeiro da área com provas formais de segurança. Nele, é apresentando uma abordagem de busca por varredura sequencial, sua ideia central consiste em cifrar cada palavra do documento por meio de um XOR entre a palavra já cifrada e uma sequência de bits pseudo-aleatórios com uma estrutura especial. Essa estrutura especial é o que permite ao servidor verificar se uma determinada palavra-chave está presente nos dados cifrados, sem decifrar o documento.

A motivação dessa escolha se deve à simplicidade das operações, que permitem acompanhar a lógica de segurança passo a passo, e às limitações que a construção apresenta e que motivaram décadas de pesquisa posterior. O restante do trabalho apresenta a teoria do método, um exemplo com valores reais, detalhes de implementação e três experimentos que verificam corretude, desempenho e o vazamento de informação característico da abordagem.

2 Detalhamento Teórico

Será considerado o cenário de *searchable symmetric encryption* (SSE) com busca de uma palavra-chave (*single keyword search*). Como introduzido anteriormente, o esquema escolhido será o de *scan* sequencial proposto por Song et al. (2000), assim como mostrado na Figura 1, que será detalhado nesta Seção.

Figura 1 – Esquema escolhido.



Fonte: Song et al. (2000)

2.1 Modelo

O esquema envolve três partes. O *dono dos dados* guarda as chaves criptográficas, cifra os documentos antes de enviá-los e é o único capaz de gerar tokens de busca. O *servidor* armazena os documentos cifrados e executa buscas quando recebe um token, mas nunca tem acesso às chaves. *Usuários autorizados* podem solicitar tokens ao dono para realizar consultas sem precisar ver o conteúdo.

O modelo de ameaça adotado é o do servidor honesto mas curioso (*honest-but-curious*): o servidor segue o protocolo corretamente, mas registra e analisa tudo o que recebe na tentativa de extrair informações adicionais. Não se pressupõe que ele vai modificar resultados ou recusar buscas; apenas que vai tentar aprender o máximo que puder a partir dos dados que passam por suas mãos. Esse modelo captura o risco real de serviços de nuvem que, embora confiáveis operacionalmente, podem ser curiosos quanto ao conteúdo armazenado.

2.2 O Esquema de Song, Wagner e Perrig (2000)

O texto plano será representado por uma sequência de l palavras W_1 até W_l . Todas as palavras do documento devem possuir o mesmo tamanho e ocupar um bloco de tamanho fixo; palavras mais curtas que esse tamanho podem ser preenchidas e palavras mais longas podem ser divididas (SONG et al., 2000). Esse esquema funciona de forma probabilística através de um parâmetro m , que deve ser escolhido em função do tamanho de uma palavra. Nesse sentido, esse esquema pode gerar correspondências que são falsos positivos. No entanto, é possível controlar o erro gerado através do parâmetro m , sendo a probabilidade de um erro em uma posição é de $\frac{1}{2^m}$ (SONG et al., 2000).

Para que a busca em dados cifrados funcione, a técnica precisa cumprir dois objetivos ao mesmo tempo. A mesma palavra deve parecer diferente em cada posição do documento, para que o servidor não consiga identificar padrões só de olhar para o texto cifrado. Ao mesmo tempo, um único token de busca deve ser capaz de encontrar todas as ocorrências dessa palavra, independentemente de quantas vezes ela aparece. Esses dois objetivos parecem contraditórios à primeira vista, mas a técnica consegue satisfazer ambos.

A ideia central é separar a cifragem em duas camadas. A primeira é determinística: a função E , com chave k'' , sempre produz o mesmo resultado para a mesma palavra. Isso significa que $E(k'', \text{"the"})$, por exemplo, é idêntico em todas as posições onde "the" aparece, e esse valor é chamado de X_i . É justamente essa consistência que permite ao token de busca encontrar todas as ocorrências. A segunda camada existe para ocultar X_i do servidor. Para cada posição i , o esquema gera um valor aleatório S_i diferente e o usa para cobrir X_i com uma operação XOR. O que o servidor armazena, portanto, é apenas o resultado desse XOR.

A partir dessa ideia, o algoritmo de cifragem funciona da seguinte forma. O conjunto de chaves é gerado aleatoriamente e compõe três valores: k_{seed} (semente do gerador de sequência), k' (chave de derivação) e k'' (chave da cifra determinística). Para cifrar cada palavra W_i , primeiro calcula-se $X_i = E(k'', W_i)$, que tem n bits. Esse valor é dividido ao meio: L_i são os primeiros $n-m$ bits e R_i são os últimos m bits. A partir de L_i , deriva-se uma chave específica para a posição: $k_i = f(k', L_i)$. Em paralelo, o gerador de sequência produz S_i a partir de k_{seed} indexado pela posição i . Com k_i e S_i , calcula-se um verificador $P_i = F(k_i, S_i)$. O bloco de cobertura é $T_i = S_i \| P_i$, com o mesmo tamanho de X_i , e o ciphertext final é $C_i = X_i \oplus T_i$.

Para buscar uma palavra W , o usuário calcula $X = E(k'', W)$ e $k = f(k', L)$, onde L é a metade esquerda de X , e envia o par (X, k) ao servidor. Esse par é o trapdoor: contém exatamente o que o servidor precisa para realizar a busca, mas não revela a palavra buscada, pois inverter a cifra E sem a chave k'' é computacionalmente inviável.

O servidor recebe (X, k) e percorre cada bloco C_i . Para cada um, calcula $V = C_i \oplus X$ e verifica se $F(k, V_{esq})$ é igual a V_{dir} , onde V_{esq} e V_{dir} são as metades esquerda e direita de V . Quando a palavra na posição i é a palavra buscada, $X_i = X$, e o XOR cancela: V

resulta em $T_i = S_i \| P_i$. Como $k = k_i$ nesse caso, a verificação $F(k, S_i) = P_i$ passa. Quando a palavra é diferente, o XOR não cancela, V não tem estrutura e a verificação falha com probabilidade $1 - 1/2^m$. Com $m = 64$, essa probabilidade é praticamente 1, tornando os falsos positivos desprezíveis.

A decifração não envolve o servidor. O usuário regenera a sequência S_i a partir de k_{seed} (o gerador é determinístico), desfaz o XOR para recuperar X_i e aplica $E^{-1}(k'', X_i)$ para obter a palavra original.

2.3 Exemplo Concreto

Para tornar o funcionamento mais concreto, os algoritmos foram aplicados ao documento ["the", "cat", "sat", "on", "the", "mat"], com $n = 128$ bits e $m = 64$ bits. Os valores abaixo foram produzidos pela execução direta do código implementado e estão representados em hexadecimal.

KeyGen gerou as seguintes chaves:

```
k_seed = ebcaa1a50d8dda82 5592931d32ecf516
k'      = 6dfc6551bc6c8548 fb141965eaccfd3e
k''     = f638607a7f6d3df6 e12c3454d442f149
```

Encrypt para a palavra “cat” na posição 1. A palavra é convertida para um bloco de 16 bytes com bytes nulos como padding e a cifragem segue os passos descritos na Seção 2.2:

```
W_1 = 6361740000000000 0000000000000000 ("cat" + 13 bytes 0x00)

X_1 = E(k'', W_1) = c72fe676b0f5cca5 47bc601746ac02d5
L_1 = X_1[:8]      = c72fe676b0f5cca5
R_1 = X_1[8:]      = 47bc601746ac02d5

S_1 = G(k_seed)[1] = 4a854df46c69f653
k_1 = f(k', L_1)   = 87b25aa5b62cec8a 596fc66d885c5786
P_1 = F(k_1, S_1)  = 9a3adc38b890a198
T_1 = S_1 || P_1   = 4a854df46c69f653 9a3adc38b890a198

C_1 = X_1 XOR T_1  = 8daaab82dc9c3af6 dd86bc2ffe3ca34d
```

Aplicando o mesmo processo a todas as seis palavras, obtêm-se os seguintes blocos cifrados:

```
C[0] "the" -> eb6de9d830b6a889 59f9b4b52d90b86a
C[1] "cat" -> 8daaab82dc9c3af6 dd86bc2ffe3ca34d
C[2] "sat" -> b1915b9b29933112 fe2a8562327f331d
C[3] "on"  -> 11a16a04a238abb0 23bfc06d267c9b6b
```

C[4]	"the"	->	170b562afc654a00 881511d34d313758
C[5]	"mat"	->	ff74fda8fddf9d85 ff09a6f626a3c531

Trapdoor para buscar “cat”. O usuário calcula $X = E(k'', \text{“cat”})$ e $k = f(k', X[:8])$ e envia o par ao servidor:

X	=	c72fe676b0f5cca5 47bc601746ac02d5
k	=	87b25aa5b62cec8a 596fc66d885c5786

Search: o servidor aplica $V = C_i \oplus X$ e verifica $F(k, V[:8]) = V[8:]$ em cada bloco. Retorna a posição 1 para “cat”, as posições 0 e 4 para “the”, e lista vazia para “dog”, que não existe no documento. Os valores gerados neste exemplo também serão usados na próxima seção para ilustrar as propriedades de segurança.

2.4 Propriedades de Segurança

Duas propriedades de segurança ficam evidentes a partir do exemplo anterior.

A primeira é o **sigilo provável**. A palavra “the” aparece nas posições 0 e 4, mas os respectivos blocos cifrados são completamente diferentes:

C[0]	=	eb6de9d830b6a889 59f9b4b52d90b86a
C[4]	=	170b562afc654a00 881511d34d313758

Isso acontece porque cada posição recebe um S_i diferente gerado pelo PRG. Como $C_i = X_i \oplus T_i$ e T_i depende de S_i , o resultado final é distinto mesmo que X_i seja o mesmo nas duas posições. O servidor não consegue saber, apenas olhando para o ciphertext, que $C[0]$ e $C[4]$ correspondem à mesma palavra.

A segunda é a **consulta oculta**. O trapdoor enviado ao servidor é o par (X, k) , onde $X = E(k'', \text{“cat”})$. Sem a chave k'' , não é possível recuperar “cat” a partir de X . O servidor consegue apenas verificar se um determinado bloco casa com o trapdoor; a palavra buscada, em si, permanece desconhecida.

Essas duas propriedades garantem privacidade do conteúdo dos documentos e das consultas, mas o esquema não oferece uma terceira propriedade que frequentemente se deseja: a ocultação do padrão de acesso.

2.5 Limitações

Toda vez que o servidor executa uma busca, ele observa quais posições são retornadas. Ao longo de muitas consultas, esse histórico acumula informação: palavras frequentes no documento retornam mais posições a cada busca, e palavras raras retornam menos. Um servidor honesto-mas-curioso pode analisar essa distribuição e inferir, com boa precisão, a frequência relativa das palavras no documento, sem decifrar nenhum bloco. Isso é o que

se chama de vazamento por padrão de acesso (*access pattern leakage*), e é demonstrado experimentalmente na Seção 4.3.

A construção também é estática: a sequência $S_1, \dots, S_l$ é gerada uma única vez a partir de k_{seed} no momento da cifragem. Adicionar ou remover palavras exigiria re-cifrar o documento inteiro. Esquemas mais recentes, como o de Curtmola et al. (2006), citado em (WANG et al., 2016), resolvem esse problema usando índices cifrados que suportam atualizações, mas com maior custo de armazenamento.

3 Implementação

3.1 Primitivas e Decisões de Projeto

O esquema foi implementado em Python 3 usando a biblioteca *cryptography*. Cada elemento abstrato do paper corresponde a uma primitiva concreta, listada no Quadro 1.

Quadro 1: Mapeamento de primitivas criptográficas

Símbolo	Função no esquema	Primitiva utilizada
E / E^{-1}	Cifra determinística das palavras	AES-128 ECB
f	Derivação de k_i a partir de L_i	HMAC-SHA256, 16 bytes
F	PRF de verificação no servidor	HMAC-SHA256, 8 bytes
G (PRG)	Geração dos valores S_i	AES-128 CTR, IV = 0

Fonte: Elaborado pelos autores.

A escolha do AES-ECB para E é intencional: a técnica requer que a cifragem seja determinística, ou seja, que a mesma palavra sempre gere o mesmo X_i , independentemente da posição. Sem isso, o trapdoor não conseguiria encontrar todas as ocorrências de uma palavra, pois cada X_i seria diferente mesmo para palavras iguais. Outros modos, como CBC ou GCM, introduzem aleatoriedade e não podem ser usados aqui por essa razão.

O PRG foi implementado com AES-CTR e IV zero. Isso garante que a sequência $S_1, \dots, S_l$ seja reproduzível a partir de k_{seed} : basta reiniciar o gerador com a mesma chave e o mesmo IV para obter exatamente os mesmos valores, o que é necessário para que a decifração funcione sem armazenar os valores S_i .

Os parâmetros foram definidos como $n = 128$ bits (16 bytes, alinhado ao bloco do AES) e $m = 64$ bits. Palavras menores que 16 bytes recebem bytes nulos como padding; palavras maiores são truncadas. A taxa de falsos positivos na busca é $1/2^{64}$ por posição, o que na prática é nula para qualquer tamanho de documento.

3.2 Pseudocódigo

Os Algoritmos 3.1 a 3.5 apresentam o pseudocódigo dos cinco algoritmos do esquema.

Listing 3.1 – KeyGen

```
Entrada: parametro de seguranca
Saida:   K = (k_seed, k', k'')

1. k_seed <- bits_aleatorios(n)
2. k'     <- bits_aleatorios(n)
3. k''    <- bits_aleatorios(n)
4. retornar K
```

Listing 3.2 – Encrypt

```

Entrada: K = (k_seed, k', k''), D = (W_1, ..., W_l)
Saida: C = (C_1, ..., C_l)

1. S[1..l] <- PRG(k_seed, l)           // l valores de (n-m) bits
2. para i = 1 ate l:
3.     X_i <- E(k'', W_i)              // cifra deterministica
4.     L_i <- X_i[1 .. n-m]
5.     k_i <- f(k', L_i)               // chave especifica da posicao
6.     T_i <- S[i] || F(k_i, S[i])    // bloco de cobertura
7.     C_i <- X_i XOR T_i
8. retornar C

```

Listing 3.3 – Trapdoor

```

Entrada: K = (k_seed, k', k''), W (palavra a buscar)
Saida: T_W = (X, k)

1. X <- E(k'', W)
2. L <- X[1 .. n-m]
3. k <- f(k', L)
4. retornar T_W = (X, k)

```

Listing 3.4 – Search (executado pelo servidor)

```

Entrada: C = (C_1, ..., C_l), T_W = (X, k)
Saida: lista de posicoes com correspondencia

1. posicoes <- lista vazia
2. para i = 1 ate l:
3.     V <- C_i XOR X
4.     s <- V[1 .. n-m]
5.     chk <- V[n-m+1 .. n]
6.     se F(k, s) = chk:
7.         adicionar i a posicoes
8. retornar posicoes

```

Listing 3.5 – Decrypt

```

Entrada: K = (k_seed, k', k''), C = (C_1, ..., C_l)
Saida: D = (W_1, ..., W_l)

1. S[1..l] <- PRG(k_seed, l)
2. para i = 1 ate l:

```

```
3.      L_i <- C_i[1..n-m]      XOR  S[i]
4.      k_i <- f(k', L_i)
5.      R_i <- C_i[n-m+1..n]    XOR  F(k_i, S[i])
6.      X_i <- L_i || R_i
7.      W_i <- E_inverso(k'', X_i)
8.  retornar D
```

4 Resultados Experimentais

Os três experimentos a seguir verificam, respectivamente, a corretude da implementação, o comportamento de desempenho previsto pela teoria e a principal vulnerabilidade da técnica. Todos foram executados com o código presente no repositório do projeto.

4.1 Corretude

O objetivo deste experimento é verificar que os cinco algoritmos produzem resultados corretos em situações típicas de uso. Foi criado um documento de 300 palavras escolhidas aleatoriamente de um vocabulário fixo, inserindo a palavra “target” manualmente nas posições 42, 150 e 275. Após a cifragem, cinco buscas foram realizadas cobrindo os casos mais relevantes do esquema, cujos resultados estão no Quadro 2.

Quadro 2: Resultados do experimento de corretude

Caso de teste	Esperado	Obtido	Status
“target” (3 ocorrências)	[42, 150, 275]	[42, 150, 275]	OK
“qwerty” (inexistente)	[]	[]	OK
“cloud” (múltiplas)	todas as pos.	todas as pos.	OK
Decifração (300 palavras)	doc. original	doc. original	OK
Trapdoor com chave errada	[]	[]	OK

Fonte: Elaborado pelos autores.

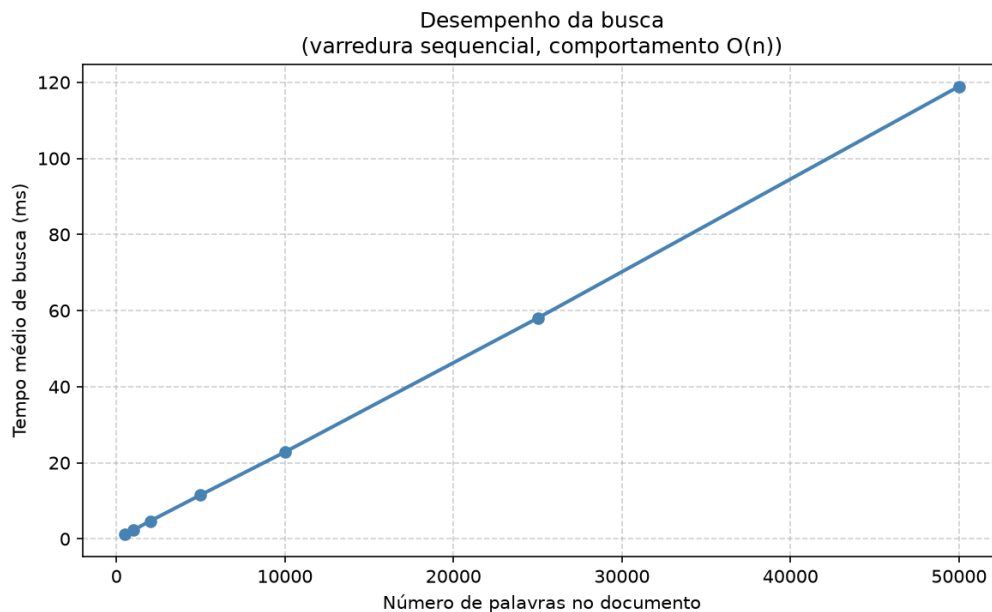
O quinto caso verifica o isolamento entre chaves diferentes. Um segundo conjunto de chaves K_2 foi gerado, o mesmo documento foi cifrado com K_2 , e tentou-se buscar nesse ciphertext usando um trapdoor gerado com a chave original K . O resultado foi uma lista vazia, confirmando que trapdoors de uma chave não produzem correspondências válidas em ciphertexts cifrados com outra chave.

4.2 Desempenho

O segundo experimento verifica que o tempo de busca cresce linearmente com o número de palavras, como previsto pela análise teórica de complexidade $O(n)$ de Song et al. (2000). Para cada tamanho testado, o documento foi cifrado e o tempo médio de cinco execuções da busca foi medido. A busca foi sempre realizada por uma palavra ausente do documento, de forma a forçar a varredura completa sem interrupção antecipada, correspondendo ao pior caso.

Os números mostram o comportamento linear: de 500 palavras (cerca de 1.2 ms) a 50.000 palavras (cerca de 120 ms), o tempo cresce proporcionalmente ao tamanho do documento. O custo por iteração é uma operação XOR de 16 bytes seguida de um cálculo

Figura 2 – Tempo médio de busca em função do número de palavras



Fonte: Elaborado pelos autores.

de HMAC-SHA256. Como não há estrutura de índice, o servidor não tem como saber em quais blocos está a palavra buscada sem varrer todos.

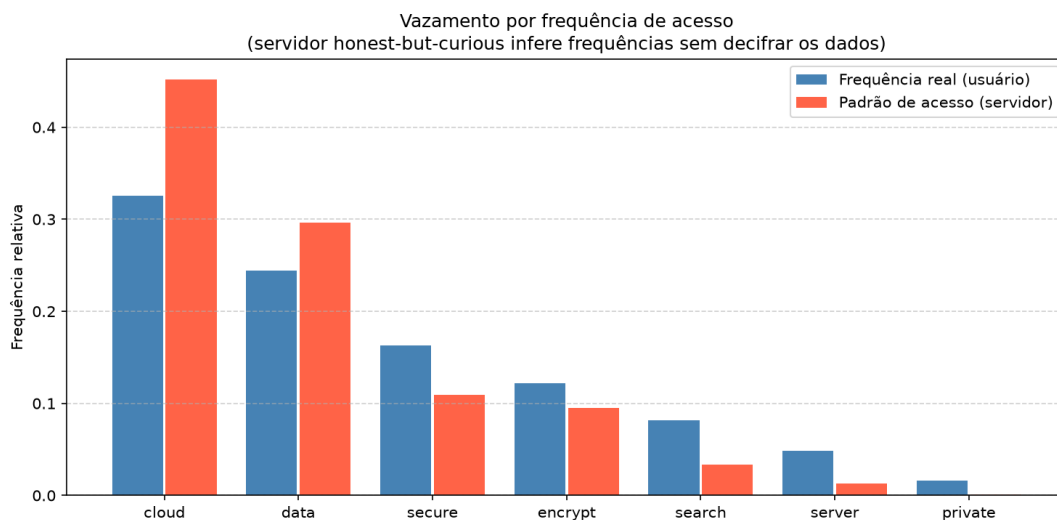
4.3 Vazamento por Frequência de Acesso

O terceiro experimento demonstra a limitação de segurança descrita na Seção 2.5. O objetivo é mostrar que um servidor honesto-mas-curioso, sem decifrar nenhum dado, consegue inferir a frequência relativa das palavras no documento apenas observando os padrões de busca ao longo do tempo.

Foi criado um vocabulário de sete palavras com frequências propositalmente desiguais: *cloud* (40 ocorrências), *data* (30), *secure* (20), *encrypt* (15), *search* (10), *server* (6) e *private* (2). O documento foi cifrado e, em seguida, simulou-se um usuário realizando 300 buscas, escolhendo qual palavra buscar com probabilidade proporcional à sua frequência real. O servidor registrou o número de posições retornadas em cada consulta e acumulou esse total por palavra ao longo de todas as 300 buscas.

A Figura 3 mostra que o ranking inferido pelo servidor a partir dos padrões de acesso reflete com precisão a frequência real das palavras no documento. Palavras mais frequentes retornam mais posições a cada busca e, ao longo de muitas consultas, acumulam muito mais acessos no histórico do servidor. Assim, o servidor consegue inferir a distribuição de frequências do documento sem ter lido nenhuma palavra. O resultado reforça o que Song et al. (2000) já reconheciam: a técnica é vulnerável a ataques estatísticos baseados em

Figura 3 – Frequência real versus padrão de acesso observado pelo servidor



Fonte: Elaborado pelos autores.

frequência, e isso foi um dos principais motivadores para o desenvolvimento de métodos SSE mais robustos nas décadas seguintes.

5 Conclusão

O trabalho implementou o esquema de busca em dados cifrados de Song et al. (2000) em Python 3, usando AES-128 ECB como cifra determinística, HMAC-SHA256 como função pseudo-aleatória e AES-128 CTR como gerador de sequência. Os cinco algoritmos do esquema foram implementados, verificados com um mini-exemplo concreto e avaliados em três experimentos distintos.

Os experimentos confirmaram as propriedades centrais da implementação. A corretude foi confirmada em cinco cenários: busca de palavras com ocorrências conhecidas, busca de palavra inexistente, múltiplas ocorrências, decifração completa e isolamento entre chaves. O desempenho seguiu a complexidade $O(n)$: o tempo de busca cresce proporcionalmente ao número de palavras, sem atalhos possíveis pela ausência de estrutura de índice. O experimento de vazamento mostrou que um servidor honesto-mas-curioso consegue recuperar a distribuição de frequências do documento apenas observando os padrões de acesso ao longo de 300 buscas.

O trabalho de Song et al. (2000) estabeleceu as bases formais da área e suas limitações impulsionaram a pesquisa seguinte. A estaticidade, que não permite atualizar o documento sem re-cifrar tudo, e o vazamento por padrão de acesso foram os dois problemas centrais. Construções baseadas em índices cifrados, discutidas em (WANG et al., 2016), resolvem ambas as questões, porém com maior custo de armazenamento e processamento. O experimento de vazamento desenvolvido aqui mostra por que esses avanços foram necessários.

Declaração sobre uso de inteligência artificial. Ferramentas de inteligência artificial foram utilizadas principalmente como auxílio na revisão e na formatação do texto. O conteúdo técnico, as decisões de projeto, a implementação dos algoritmos e a execução dos experimentos são, em sua maior parte, de autoria dos integrantes do grupo.

Referências

- SONG, D. X.; WAGNER, D.; PERRIG, A. Practical techniques for searches on encrypted data. In: *Proceedings of the 2000 IEEE Symposium on Security and Privacy*. Berkeley, CA, EUA: [s.n.], 2000. p. 44–55.
- WANG, Y.; WANG, J.; CHEN, X. Secure searchable encryption: a survey. *Journal of Communications and Information Networks*, v. 1, n. 4, p. 52–65, 2016.